UCab – Cab Booking Platform
Project Documentation

1. Introduction

Project Title:
UCab – Cab Booking Platform

Team Members:
Team Leader – Asad Ali
Team Member – Ayushi Gupta
Team Member – Mahak Vishwakarma
Team Member – Ganguli Kumar Yadav

---

2. Project Overview

Purpose

UCab is a web-based cab booking platform developed using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**. The purpose of the project is to provide a digital system where passengers can request rides and drivers can accept ride requests through a centralized platform. The system simplifies ride management and improves accessibility to transportation services.

Features

- User Registration and Login
- Secure Authentication using JWT
- Ride Request Creation
- Driver Ride Acceptance
- Ride Status Updates
- Trip History Management
- Database Storage of Users and Trips
- Admin Monitoring Capability

---

3. Architecture

Frontend

The frontend is developed using **React.js**. It provides the user interface for riders and drivers to interact with the system.

Main responsibilities of the frontend include:

- Displaying user interfaces for login and registration
- Allowing riders to request rides
- Allowing drivers to manage ride requests
- Communicating with backend APIs through HTTP requests
- Displaying trip information and ride status updates

React components are used to modularize the UI and improve maintainability.

Backend

The backend is implemented using **Node.js with Express.js**.

The backend handles:

- Authentication and authorization
- Processing ride requests
- Managing driver and rider interactions
- Handling trip creation and updates
- Managing communication between frontend and database
- Providing RESTful APIs to the frontend

Express middleware is used for routing, request handling, and API management.

Database

The application uses **MongoDB** as the database.

MongoDB stores data related to:

- Users (riders and drivers)
- Ride requests
- Trip records

- Payment data (if implemented)
- User feedback and ratings

Mongoose is used as the Object Data Modeling (ODM) library to interact with MongoDB collections.

---

4. Setup Instructions

Prerequisites

Before running the project, install the following:

- Node.js (v16 or higher)
- MongoDB
- Git
- npm (Node Package Manager)

Installation

Step 1 – Clone the repository

```
git clone https://github.com/itsasad786/Ucab
```

Step 2 – Navigate to the project folder

cd Ucab

Step 3 – Install dependencies

Client:

```
cd client
npm install
```

Server:

```
cd server
npm install
```

Step 4 – Configure Environment Variables

Create a .env file in the server directory and add:

```
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
PORT=5000
```

---

5. Folder Structure

Client (Frontend)

The **client** folder contains the React application.

Typical structure:

-client/
-src/
-components/
-pages/
-App.js
-index.js

Components contain reusable UI elements while pages represent major application screens.

Server (Backend)

The **server** folder contains the Node.js backend.

Typical structure:

-server/
-controllers/
-models/
-routes/
-middleware/
-server.js

- **Controllers** handle business logic
- **Models** define MongoDB schemas
- **Routes** define API endpoints
- **Middleware** handles authentication and request processing

---

6. Running the Application

To run the application locally, start both frontend and backend servers.

Frontend

Navigate to the client directory:

```
cd client
npm start
```

The React application will start at:

http://localhost:5173

Backend

Navigate to the server directory:

```
cd server
npm start
```

The backend server will start at:

http://localhost:5000

---

7. API Documentation

The backend exposes several REST API endpoints.

User Authentication

POST /api/auth/register

Registers a new user.

Request Example

```
{
"name": "John",
"email": "john@example.com",
"password": "123456"
}
```

POST /api/auth/login

Logs a user into the system.

Request Example

```
{
"email": "john@example.com",
"password": "123456"
}
```

Ride Management

POST /api/rides/request

Creates a ride request.

GET /api/rides

Fetches available rides.

POST /api/rides/accept

Driver accepts a ride request.

GET /api/rides/history

Returns trip history for the user.

---

8. Authentication

Authentication in UCab is handled using **JWT (JSON Web Tokens)**.

Workflow:

1. User registers or logs in.
2. The server generates a JWT token.
3. The token is sent to the client.
4. The client includes the token in future API requests.
5. Middleware verifies the token before allowing access to protected routes.

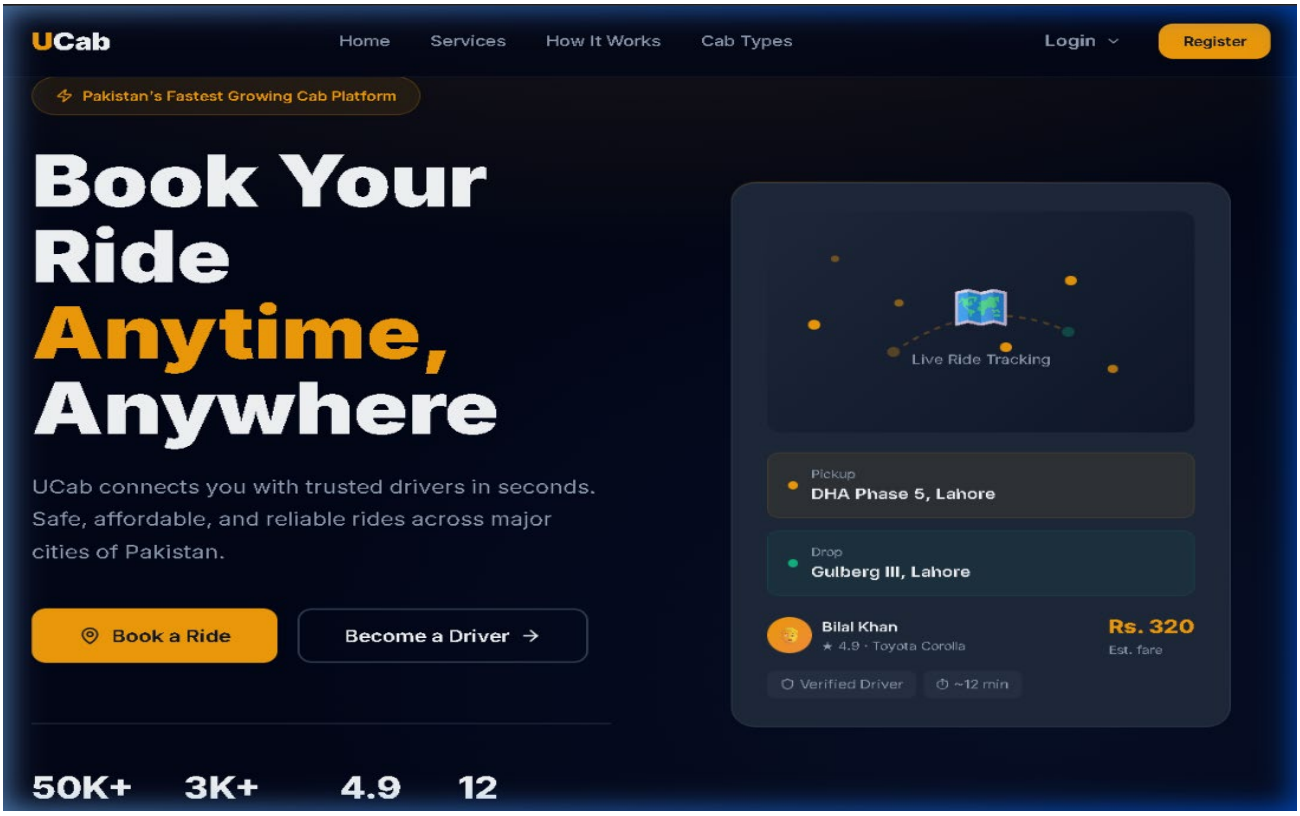
This ensures secure communication between client and server.

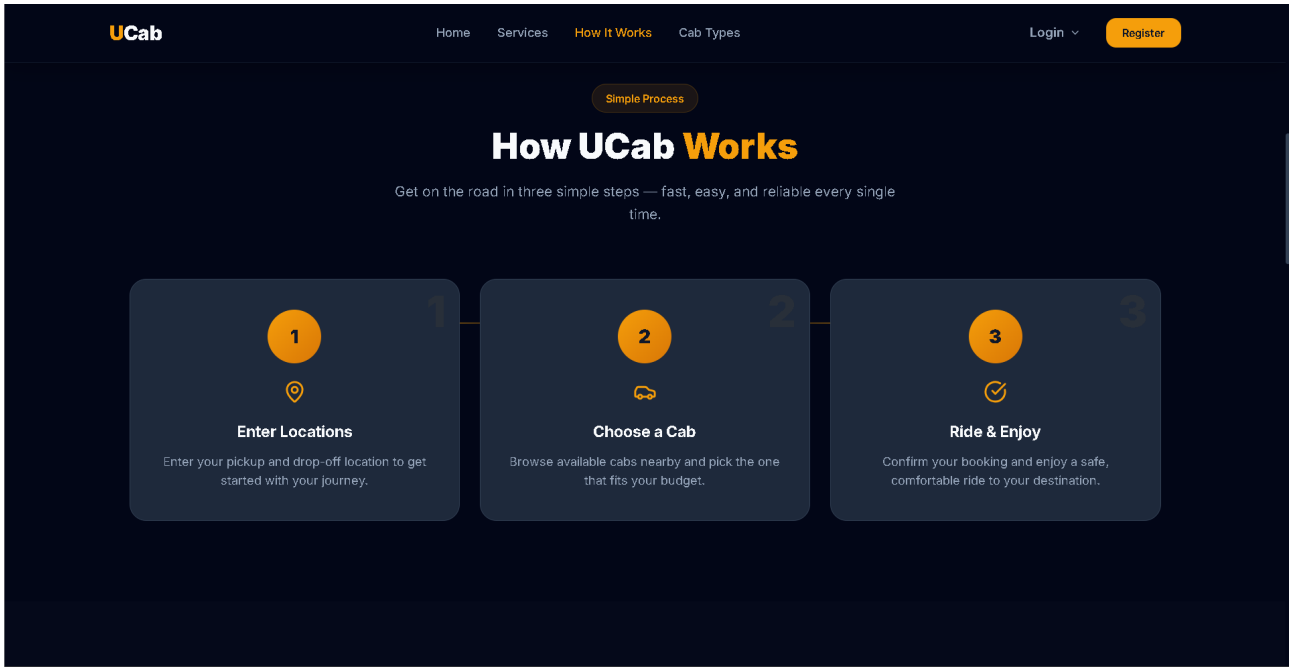
---

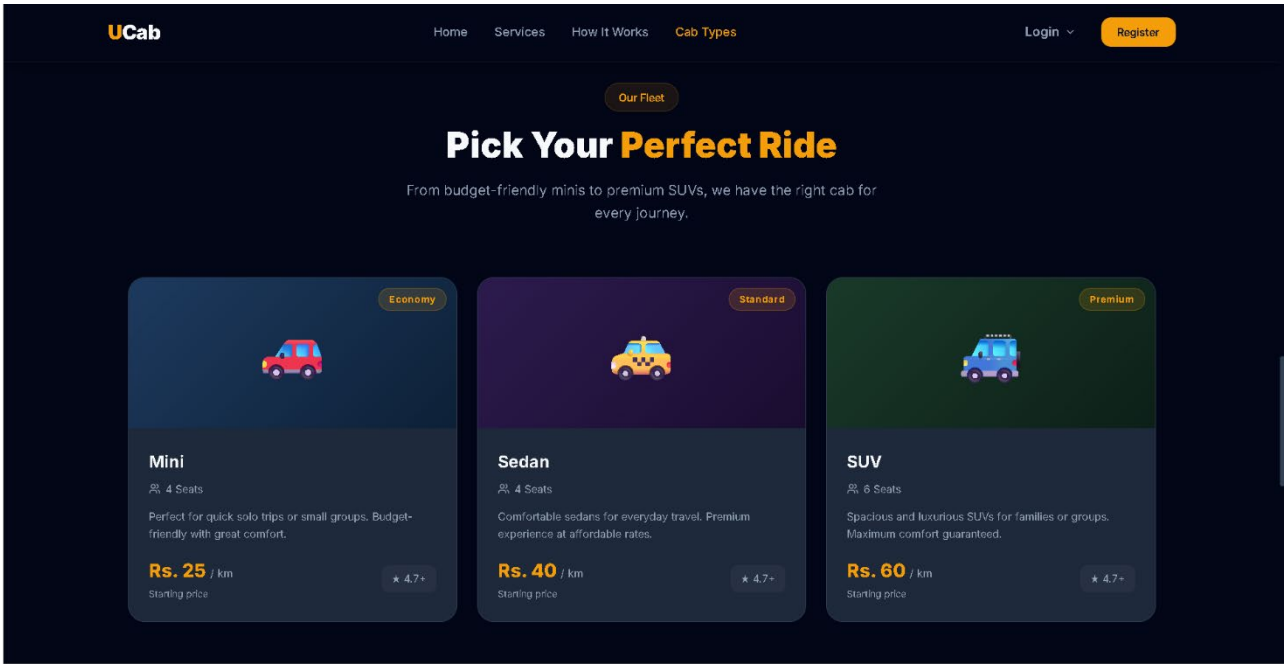
9. User Interface

The UI includes several key screens:

- Home Page
- Login Page
- Registration Page
- Ride Booking Interface
- Driver Dashboard
- Trip History Page


UCab
Home
Services
How It Works
Cab Types
Login
Register
Pakistan's Fastest Growing Cab Platform
Book Your Ride Anytime, Anywhere
UCab connects you with trusted drivers in seconds. Safe, affordable, and reliable rides across major cities of Pakistan.
Book a Ride
Become a Driver
Live Ride Tracking
Pickup
DHA Phase 5, Lahore
Drop
Gulberg III, Lahore
Bilal Khan
★ 4.9 · Toyota Corolla
Rs. 320
Est. fare
Verified Driver
~12 min
50K+
3K+
4.9
12


UCab
Home
Services
How It Works
Cab Types
Login
Register
Simple Process
How UCab Works
Get on the road in three simple steps — fast, easy, and reliable every single time.
1
Enter Locations
Enter your pickup and drop-off location to get started with your journey.
2
Choose a Cab
Browse available cabs nearby and pick the one that fits your budget.
3
Ride & Enjoy
Confirm your booking and enjoy a safe, comfortable ride to your destination.



---

10. Testing

Testing was performed to verify system functionality and performance.

Testing included:

- Functional Testing
- API Testing
- Authentication Testing
- Performance Testing

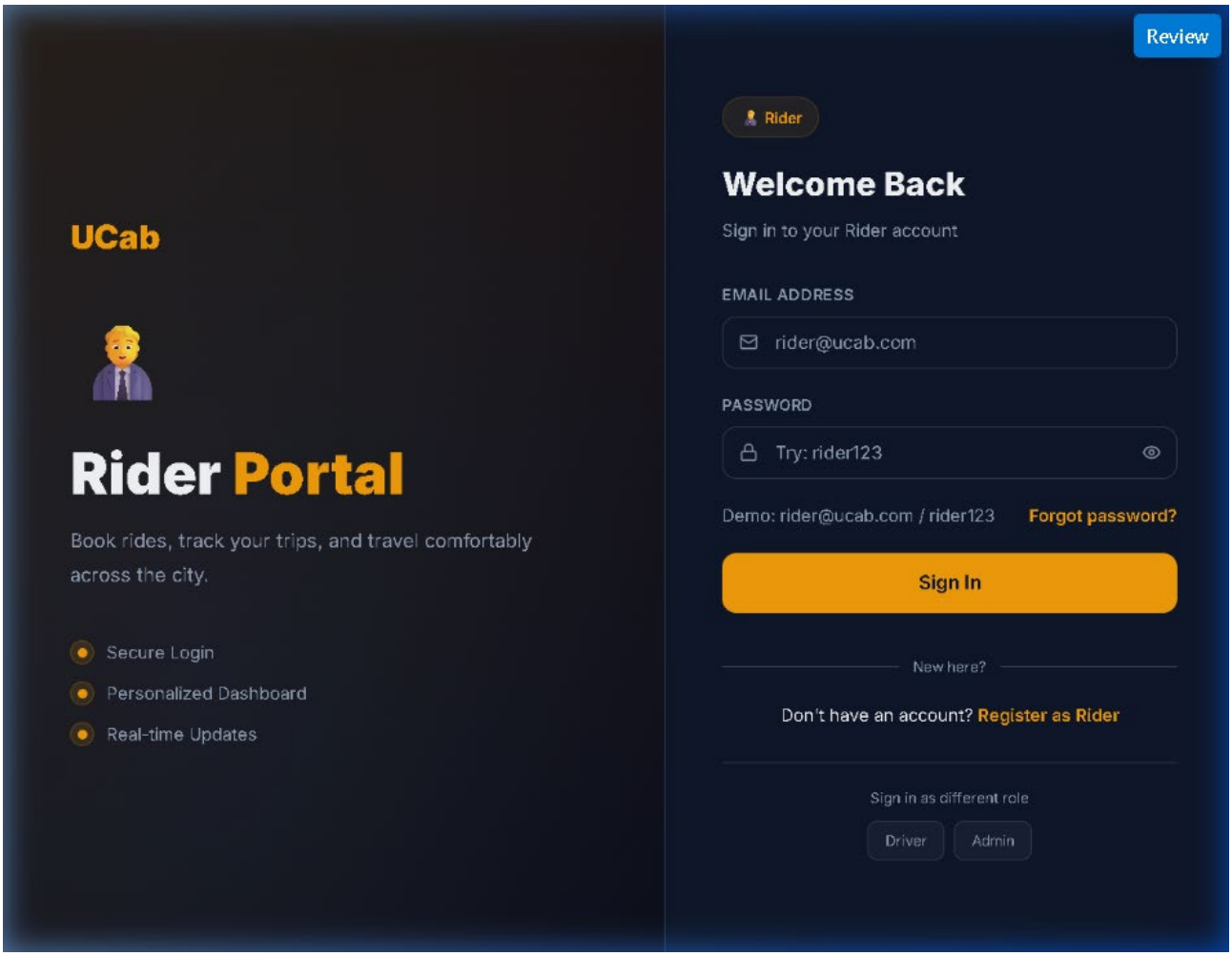
The goal was to ensure that all system features worked as expected and that the system could handle multiple user requests efficiently.

---

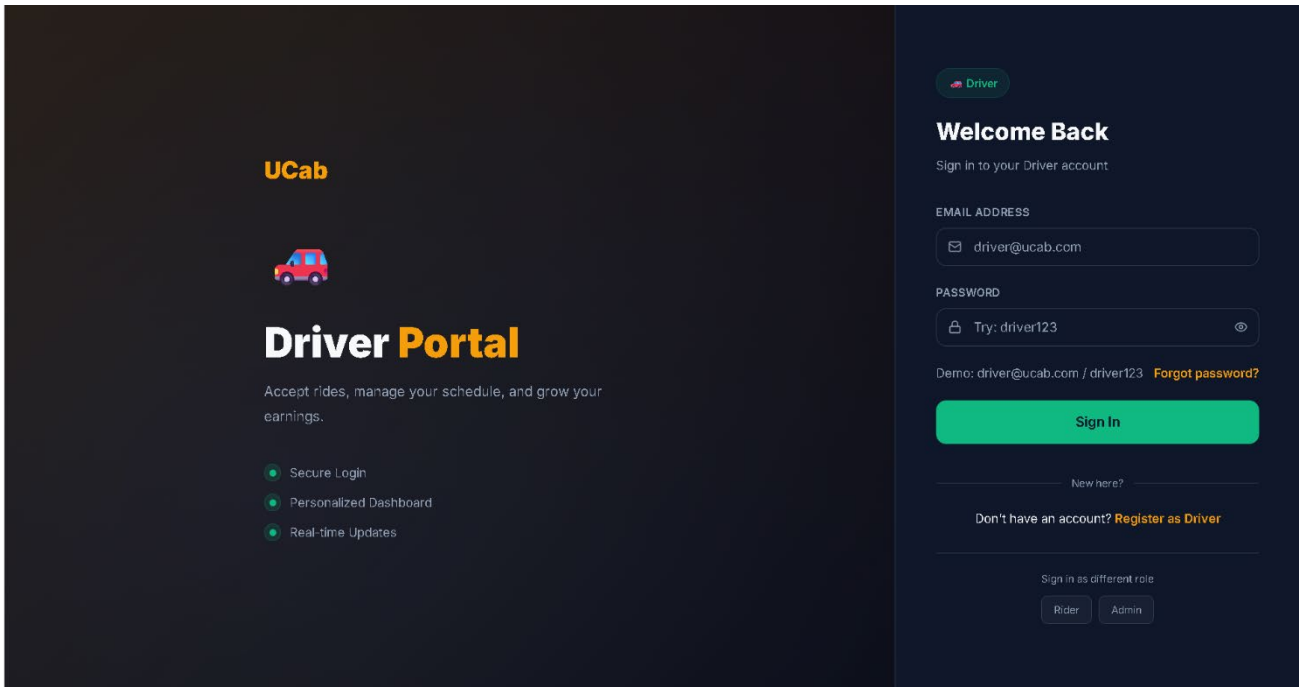
11. Screenshots or Demo

Screenshots of the following should be included:

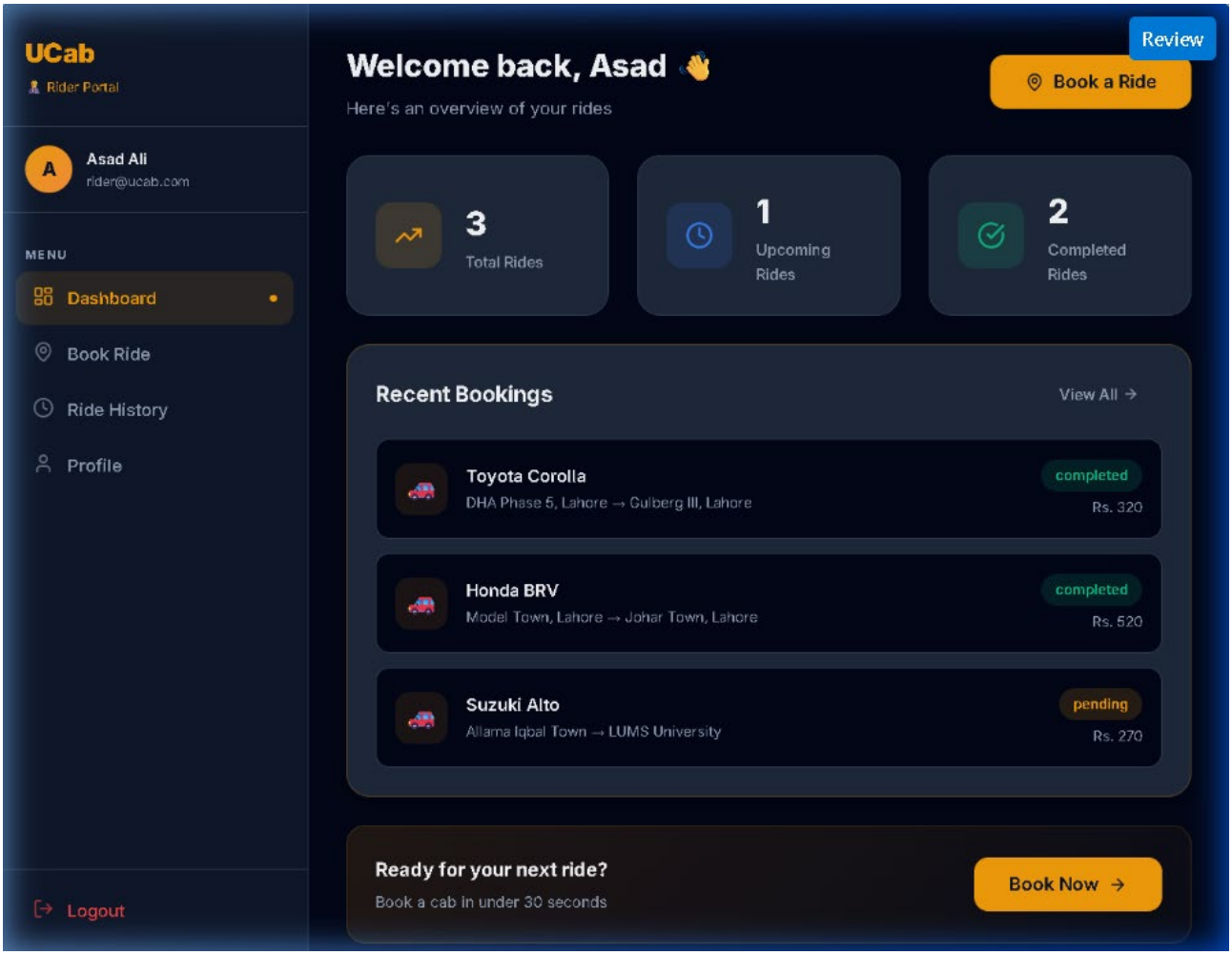
- Registration/Login Page



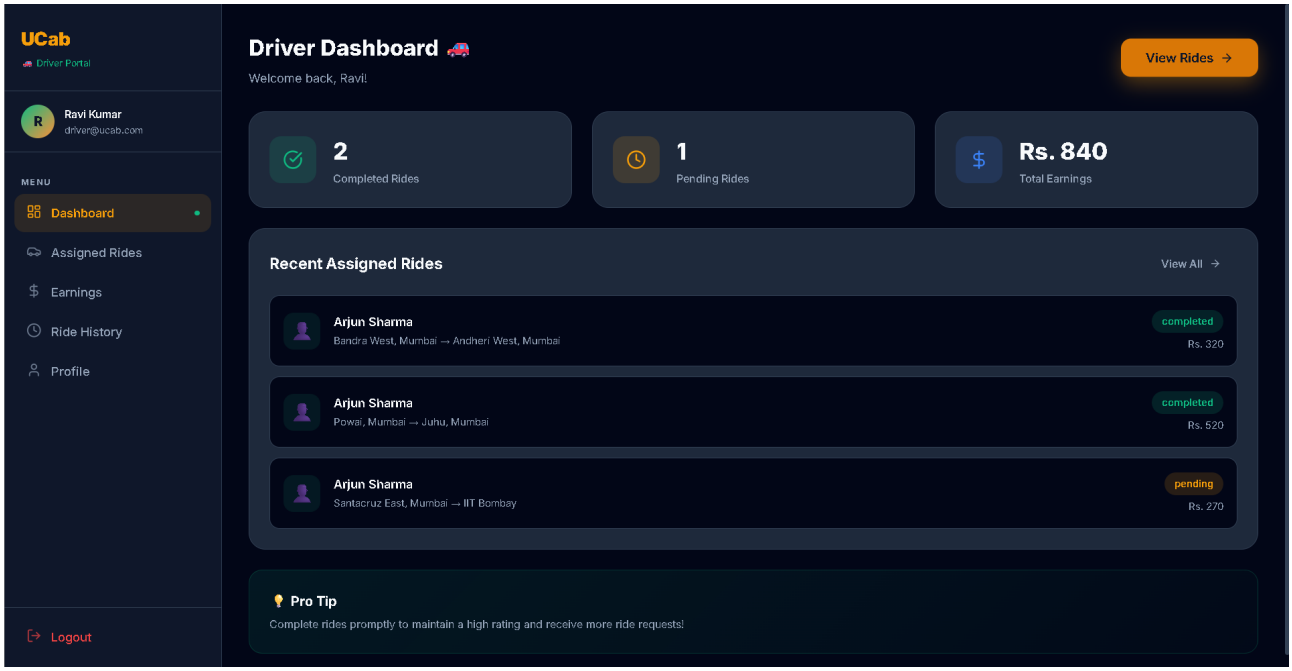
- Driver Interface



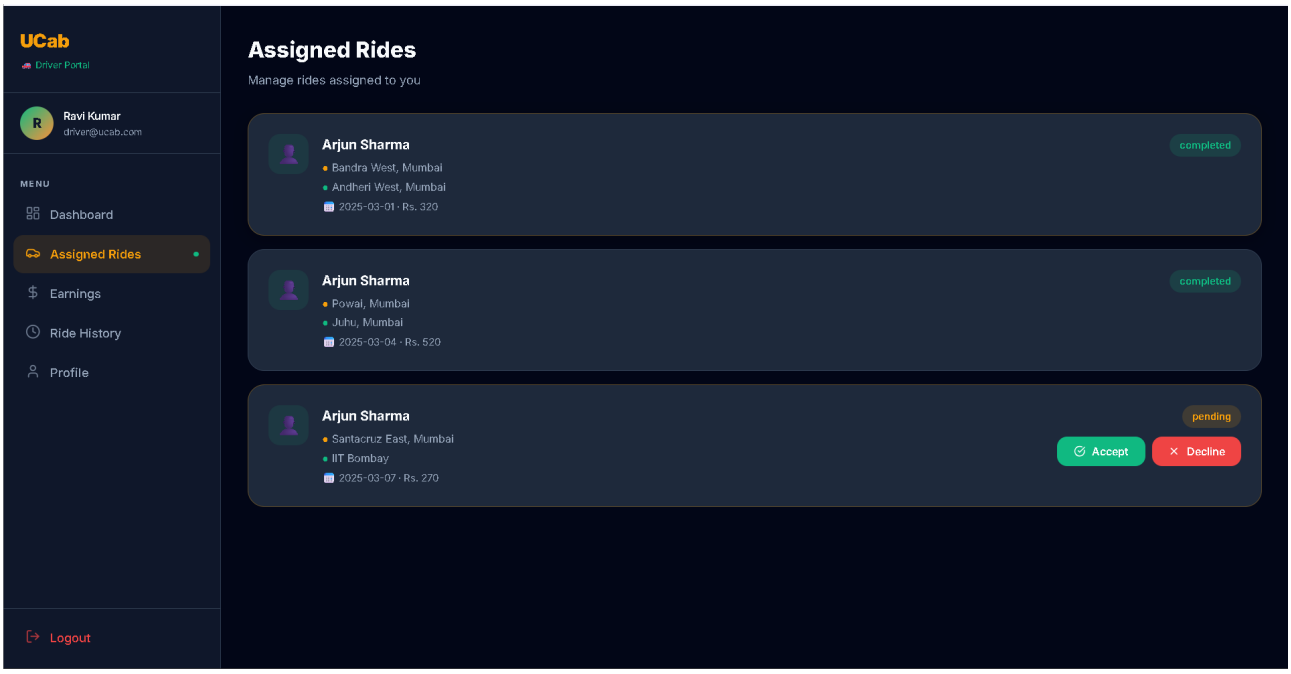
- Ride Booking Interface



- Driver Dashboard



- Trip History Page



GitHub Repository:

https://github.com/itsasad786/Ucab

---

12. Known Issues

Some minor issues identified during development include:

- UI responsiveness on small screens
- Limited real-time tracking functionality
- No integrated payment gateway yet

These issues do not prevent the core system from functioning.

---

13. Future Enhancements

Possible future improvements include:

- Mobile application for riders and drivers
- Real-time GPS tracking integration
- Online payment gateway integration
- Driver rating and review system
- Push notifications

- Advanced ride matching algorithms

These enhancements will improve usability, scalability, and system functionality.